

mersch75.lu — Webseiten-Statusbericht & Ausbau-Empfehlungen

Erstellt: 29. Juni 2026 · **Autor:** Cascade (KI-Assistenz) · **Basis:** Code-Analyse des Repos `Netjogger58/mersch75test.github.io` **Zweck:** Ehrlicher Ist-Zustand (fertig / ausbaufähig / nur Idee) der öffentlichen Vereins-Website, Bewertung von Struktur, Komplexität, Aufbau & Visuellem — plus konkrete Anregungen inkl. SEO-, Performance- & Automatisierungs-Konzept.

0. Kurzfazit (TL;DR)

mersch75.lu ist **keine Baukasten-Website, sondern eine umfangreiche, handgebaute Vereins-Plattform**: 34 HTML-Seiten, ~18.400 Zeilen HTML, ein zentrales JavaScript (`script.js` , 2.311 Zeilen) und ein großes Stylesheet (`styles.css` , 5.374 Zeilen / 111 KB). Sie läuft **kostenlos auf GitHub Pages** unter der eigenen Domain **mersch75.lu** (CNAME) — **ohne Server, ohne Build-Schritt, ohne Framework** (reines HTML/CSS/Vanilla-JS).

Das Herzstück ist die **Live-Anbindung an den Handball-Verband**: `js/flh-live-sync.js` zieht Ergebnisse & Tabellen direkt von `handball4all.de` (mit Abstand am häufigsten referenzierte Quelle — **378 Treffer** im Code) für **10 Teams/Ligen**. Dazu kommen ein **5-sprachiges i18n-System** (LU/FR/DE/EN/PT), Kontaktformulare über **Web3Forms**, ein interner **Poster-/Content-Generator** und vollständige **RGPD-Rechtstexte** (`dataprotection.html` allein 103 KB).

Größtes Risiko: **Repo-Größe & „Soft-Locks“**. Das Repository ist **851 MB groß** (davon `assets/` 193 MB, `Media/` 5,9 MB Alt-Bestand) — schwer für Git/Pages. Die internen Werkzeuge sind nur per **Klartext-Passwort im öffentlichen Quelltext** geschützt (`'mersch75'`), was ausdrücklich nur „Zufallsbesucher fernhält“, aber kein echter Schutz ist.

1. Kennzahlen (gemessen am Code)

Metrik	Wert	Anmerkung
HTML-Seiten	34	Top-Level <code>*.html</code>
HTML-Zeilen gesamt	~18.439	über alle Seiten
JavaScript-Zeilen	~3.282	<code>script.js</code> + <code>js/</code> + <code>feedback.js</code>
CSS-Zeilen	5.374	<code>styles.css</code> (111 KB)
<code>script.js</code>	2.311 Zeilen	zentrale Logik (i18n, Menü, News, Formulare)
Sprachen	5	LU · FR · DE · EN · PT (Default LU)
<code>data-i18n</code> -Marker	289	Übersetzungs-Anker in HTML
FLH-Live-Ligen	10	Männer, Frauen, U15/U13/U11/U9/U7
Externe Quellen	s. §7	dominiert von <code>handball4all.de</code>
Repo-Größe gesamt	851 MB	<code>assets/</code> 193 MB · <code>Media/</code> 5,9 MB

Einordnung: Das ist eine **große, eigenständig entwickelte Website** mit echter Datenanbindung — deutlich mehr als eine klassische Vereins-Visitenkarte.

2. Architektur & Aufbau

2.1 Schichten

```
Besucher-Browser
| statisches HTML/CSS/JS (kein Build, kein Server)
▼
GitHub Pages (Hosting) → Domain mersch75.lu (CNAME)
|
├─ script.js          (i18n, Menü, News-Carousel, Training, Formulare)
├─ js/flh-live-sync → fetch → handball4all.de (Live-Ergebnisse/Tabellen)
├─ js/*-gate.js       (Soft-Lock für interne Tools)
└─ Web3Forms-API      (Kontakt-/Anmelde-Formulare)
```

2.2 Technologie

- **Reines Frontend:** HTML5 + CSS3 + Vanilla-JavaScript, **kein Framework, kein Bundler, kein Node-Build** → maximal einfach zu hosten, läuft direkt auf GitHub Pages.
- **Styling:** ein zentrales `styles.css` (111 KB) — eigenes Design-System, „Apple-/iOS-style“ mit abgerundeten Karten, Postersektionen, Sprach-Umschalter.
- **Mehrsprachigkeit:** eigenes i18n in `script.js` über `data-i18n` / `data-i18n-attr` -Attribute, 5 Sprachen, **Lëtzebuergesch als Standard**, Auswahl im Header.
- **Schrift/Icons:** Google Fonts, Inline-SVG-Icons, teils `cdnjs` / `unpkg` für einzelne Libs.

2.3 Zentrale Bausteine in `script.js`

```
initializeSiteLanguage · initializeSiteMenu · initializeNewsCarousel (mit Autoplay) ·
initializeTrainingSchedule · initializeJoinUsForm · initializeFeedbackSystem · initializeSharedFooters ·
syncCurrentYear . → Ein gemeinsames Skript für alle Seiten, dadurch konsistentes Verhalten.
```

2.4 Live-Daten (`js/flh-live-sync.js`)

- Spricht die offizielle **handball4all-JSON-Schnittstelle** an (`if_g_json.php` , `og=95`).
- Holt **Spielpläne, Ergebnisse, Tabellen** für 10 Mannschaften, **dedupliziert** Spiele und verlinkt **SBO-Spielberichte** automatisch.
- Das ist der technisch anspruchsvollste Teil der Website.

2.5 Interne Werkzeuge

- `generator.html` (3.167 Zeilen / 216 KB): Poster-/Inhalts-Generator für Social Media & Matchdays.
- `live-center.html` (1.490 Zeilen): Match-Center / Spielzentrale.
- `hallenkarte.html` + `-editor.html` : interaktive Hallen-/Sitzplan-Funktion.
- `kees-scanner.html` : Scanner-Tool.
- Geschützt durch **Soft-Locks** (`js/generator-gate.js` , `js/events-gate.js`): Passwort-Modal, Status 7 Tage im `localStorage` .

2.6 Bewertung Aufbau

- **Stark:** keine Server-Kosten, extrem robustes Hosting, echte Live-Daten, 5 Sprachen, saubere Asset-Policy (`AGENTS.md` , `assets/shared` vs. `assets/pages/<page>`), vollständige Rechtstexte.
- **Schwächen / Schulden:**

- **Repo 851 MB** — `assets/` (193 MB) und `Alt-Media/` (5,9 MB) liegen mit in Git → langsame Clones, an der GitHub-Pages-Grenze (1 GB empfohlen).
- **Monolithische Dateien:** `script.js` (2.311 Z.), `styles.css` (111 KB), `generator.html` (216 KB) — schwer wartbar.
- **Backup-Dateien im Repo:** `generator.html.bak-logofix`, `generatorold.html`, `mersch75-legal-footer.patch` (503 KB) → sollten ins Archiv/ `.gitignore`.
- **Soft-Lock-Passwort im Klartext** (`'mersch75'`) — bewusst nur Hürde, kein Schutz.
- **Kein Build/Minify/Tests** — keine automatische Prüfung auf tote Links/Bilder (nur manuelle Validierung laut `AGENTS.md`).

3. Visuelles Konzept

- **Design-Sprache:** modern, „Apple-/iOS-style“ — große **Poster-Hero-Sektionen** (Portrait/Landscape mit Download & Teilen), abgerundete Karten, klare Sektions-Rahmen, animierter „Ball-Jump“ zum Spielplan.
- **Markenbild:** Vereins-Branding „75“ + „Zesumme Staark“, konsistente Logos (zentriert, vereinheitlicht laut letzten Commits).
- **Navigation:** Header mit Sprach-Umschalter (5 Flaggen), Burger-Menü mobil, gemeinsame Footer auf allen Seiten.
- **News-Carousel** mit Autoplay auf der Startseite.
- **Responsiv:** `viewport -Meta + <picture> / srcset` für Portrait/Landscape-Varianten → mobil optimierte Bilder.
- **Verbesserungspotenzial:**
- **Performance:** großes `styles.css` und viele Bilder → CSS aufteilen/minifizieren, Bilder konsequent als WebP + `loading="lazy"` .
- **Einheitliche Lade-/Leerzustände** für Live-Daten (Skeletons statt leerer Tabellen, wenn FLH-API langsam ist).
- **Konsistente i18n-Abdeckung:** nicht alle Seiten sind vollständig in allen 5 Sprachen gepflegt.

4. Seiten-Status: fertig / ausbaufähig / nur Idee

Legende:  **Fertig & gepflegt** ·  **Funktioniert, ausbaufähig** ·  **Mini-Stub** ·  **Internes Tool (Soft-Lock)** ·  **Idee**

4.1 **Fertig & gepflegt (Haupt-Seiten)**

Seite	Datei	~Zeilen	Inhalt
Startseite	<code>index.html</code>	564	Hero-Poster, News-Carousel, Schnellzugriff
Spielzentrale	<code>live-center.html</code>	1.490	Live-Spielplan/Ergebnisse (FLH)
Mitmachen	<code>join.html</code>	1.336	Anmeldung/Beitritt (Web3Forms)
Inside (Verein/History)	<code>inside.html</code>	1.030	Geschichte, Equipe, Superjhemp
Events	<code>events.html</code>	844	Veranstaltungen (Web3Forms)
Statistik 25/26	<code>statistics-25-26.html</code>	634	Saison-Statistiken
Rechtstexte	<code>dataprotection.html</code>	434 (103 KB)	RGPD-Datenschutz
Useful Links	<code>links.html</code>	365	Verbands-/Partner-Links
Terms / Impressum	<code>terms.html</code> / <code>impressum.html</code>	319 / –	Rechtliches

4.2 **Funktioniert, ausbaufähig**

Seite	Datei	Ausbau-Idee
News	news.html	redaktioneller Workflow statt Hand-HTML
Training	training.html	Live-Sync mit App-Trainingsplan
Galerie / Memories	gallery.html / memories.html	Lightbox, lazy-loading, Alben
Community / Contact	community.html / contact.html	mehr Self-Service, FAQ
NEXTGEN	nextgen.html	Jugend-Bereich ausbauen
Trainerstaff	trainerstaff.html	Profile, Kontakt
Hallenkarte	hallenkarte.html	Anfahrt/Parken, Karten-Integration
Statistik-Seiten Jugend	Statistics U11/U13/U15.html	Vereinheitlichung mit Live-Sync
Willkommensmappe	wellkomm-mapp.html	direkter PDF-Download (siehe App-Bericht)

4.3 **Mini-Stubs (faktisch leer)**

comite.html (660 B) · history.html (668 B) — winzige Platzhalter, Inhalt wandert offenbar nach inside.html .
Empfehlung: entweder füllen oder per Redirect auflösen.

4.4 **Interne Tools (nur Soft-Lock)**

generator.html (Poster-Generator) · hallenkarte-editor.html · kees-scanner.html · feedback-intern.html · nextgen.html -Editing. → Funktionieren, sind aber **nur per Klartext-Passwort** abgesichert (siehe §6).

4.5 **Nur Idee / Altlasten**

generatorold.html , generator.html.bak-logofix , mersch75-legal-footer.patch , diverse Python-Hilfsskripte (add_u11_news_translations.py) und Migrations-Reports unter data/ . → Entwicklungs-/Migrations-Spuren, gehören nicht in die Live-Auslieferung.

5. Komplexitätsbewertung

Dimension	Bewertung	Begründung
Funktionsumfang	Hoch	34 Seiten, Live-Daten, 5 Sprachen, interne Tools
Technische Tiefe	Mittel-Hoch	FLH-API-Sync & Generator anspruchsvoll; sonst statisch
Wartbarkeit	Mittel	Monolith-Dateien, viel Hand-HTML, Backups im Repo
Performance	Mittel	großes CSS/viele Bilder; kein Minify/Build
Sicherheit	Niedrig-Mittel	Soft-Locks im Klartext; öffentlich, daher begrenzt sensibel
SEO/Auffindbarkeit	Mittel	Title/Description vorhanden; Sitemap/strukturierte Daten ausbaubar
Hosting-Robustheit	Sehr hoch	GitHub Pages, statisch, quasi unkaputtbar
Doku/Konventionen	Hoch	AGENTS.md , Asset-Policy, Sprach-Guide

6. Sicherheit & Datenschutz (ehrlich)

- **Soft-Locks ≠ Schutz:** `generator-gate.js` / `events-gate.js` prüfen ein **Klartext-Passwort** `'mersch75'` im öffentlich lesbaren JavaScript. Das ist im Kommentar selbst so dokumentiert. → Echte interne Funktionen gehören **hinter Login in die Vereins-OS-App**, nicht auf die öffentliche Website.
- **Formulare:** `web3forms` auf `contact.html`, `events.html`, `join.html`. Der **Access-Key liegt im Quelltext** (üblich bei Web3Forms, aber: **Captcha/Spam-Schutz aktivieren**).
- **RGPD:** sehr gründlich — `dataprotection.html` (103 KB), Impressum, Terms, Verweise auf CNPD/INAPS. Stark.
- **Empfehlung:** interne Tools aus dem öffentlichen Repo entfernen oder klar als „intern, kein Schutz“ kennzeichnen; sensible Editier-Funktionen in die App verlagern.

7. Externe Abhängigkeiten (gemessen)

Quelle	Treffer	Rolle
<code>spo.handball4all.de</code>	378	Live-Ergebnisse & Tabellen (Kern)
<code>play.rtl.lu</code>	34	Video/Medien
<code>youtu.be</code> / <code>youtube.com</code>	48	eingebettete Videos
<code>facebook.com</code> / <code>instagram.com</code>	45	Social-Media-Links
<code>fonts.googleapis.com</code> / <code>gstatic</code>	32	Web-Fonts
<code>flh.lu</code> / <code>handball.lu</code>	22	Verbands-Links
<code>api.web3forms.com</code>	7	Formular-Versand
<code>cnpd.public.lu</code> / <code>inaps.public.lu</code>	15	RGPD/Behörden
<code>cdnjs</code> / <code>unpkg</code>	9	JS-Bibliotheken

Klumpenrisiko: Die Website hängt stark an **handball4all.de**. Fällt die API aus oder ändert das Format, brechen Live-Tabellen. → Caching + Fallback („zuletzt bekannter Stand“) sinnvoll.

8. Meine Anregungen (priorisiert)

8.1 Sofort / geringer Aufwand

1. **Repo entschlacken:** Backups (`.bak`, `generatorold.html`, `*.patch`) und Alt- `Media/` ins `archive/` oder `.gitignore` → schnellere Clones, weg von der 1-GB-Pages-Grenze.
2. **Bild-Performance:** alle großen Bilder als **WebP**, `loading="lazy"`, korrekte `width/height` → bessere Ladezeit & SEO.
3. **Mini-Stubs auflösen** (`comite.html`, `history.html`): füllen **oder** 301-artig per `<meta refresh>` /JS-Redirect auf `inside.html`.
4. **Spam-Schutz** bei Web3Forms-Formularen aktivieren.
5. **sitemap.xml** + **robots.txt** ergänzen (GitHub Pages kann das statisch).

8.2 Mittlerer Aufwand

1. **FLH-Sync absichern:** Ergebnisse cachen (z. B. JSON im Repo via Cron/Action) + Fallback-Anzeige bei API-Ausfall.
2. **CSS/JS aufteilen & minifizieren** (optionaler einfacher Build via GitHub Action, ohne den „statisch“-Vorteil zu verlieren).
3. **i18n-Lücken schließen:** Skript, das fehlende `data-i18n` -Keys je Sprache meldet.
4. **Strukturierte Daten** (`schema.org` SportsOrganization, Events) → bessere Google-Darstellung.
5. **Linkchecker als GitHub Action** (tote Links/Bilder automatisch melden — passt zu `AGENTS.md`).

8.3 Strategisch (Verbindung zur Vereins-OS-App)

1. **Öffentliche Read-API der App** (siehe App-Bericht) → Website zeigt **Live-Tabelle/nächste Spiele/Torschützen** direkt aus der App statt nur aus handball4all.
2. **Interne Tools in die App verlagern** (Generator-Editing, Scanner, Feedback-intern) → echter Login statt Soft-Lock.
3. **News-/Event-Pflege aus der App** generieren (App ist „Quelle“, Website ist „Schaufenster“).
4. **PWA-Funktion** (installierbar, Offline-Spielplan) für die öffentliche Seite.

9. SEO-, Performance- & Automatisierungs-Konzept

9.1 SEO

- `sitemap.xml` , `robots.txt` , kanonische URLs, Open-Graph-/Twitter-Cards je Seite, `schema.org` -Markup für Verein & Events, sprechende `alt` -Texte (mehrsprachig).

9.2 Performance

- WebP + `lazy` -Loading, CSS-Split (kritisches CSS inline), Preload der Web-Fonts, optional einfache Minify-Action.
Ziel: gute Lighthouse-Werte trotz statischem Hosting.

9.3 Automatisierung (passt zur App-Roadmap)

- **GitHub Action / n8n:** täglicher FLH-Abruf → JSON-Cache committen → Website nutzt Cache (schnell, ausfallsicher).
- **Linkchecker-Action** bei jedem Push (Validierung wie in `AGENTS.md` gefordert).
- **Auto-Deploy:** Push → Pages (bereits gegeben); zusätzlich Image-Optimierung in der Pipeline.
- **Brücke App ↔ Website:** App generiert News/Poster → committet in dieses Repo oder liefert per API → Website aktualisiert sich automatisch.

10. Risiken & nächste sinnvolle Schritte

Risiken: Repo-Größe (851 MB) nahe Pages-Grenze · Klumpenrisiko handball4all · Soft-Locks ohne echten Schutz · Monolith-Dateien & Altlasten · keine automatischen Link-/Bild-Checks.

Empfohlene nächste 5 Schritte:

1. Repo entschlacken (Backups/Alt-Media raus) + `.gitignore` schärfen.
 2. Bild-/CSS-Performance (WebP, lazy, optional Minify-Action).
 3. `sitemap.xml` / `robots.txt` / Open-Graph + Linkchecker-Action.
 4. FLH-Sync cachen + Fallback.
 5. Brücke zur Vereins-OS-App: erste **Live-Tabelle/Spiele aus der App-Read-API** auf der Startseite.
-

*Hinweis: Dieser Bericht basiert auf statischer Code-Analyse des Website-Repos vom 29.06.2026.
Zeilen-/Größenangaben sind gemessen (`wc -l` , `du` , `grep`).*